ADVANCED DATABASE SYSTEMS

AfolabiPay: Database Project Report

Oladipo Afolabi | Student ID: 301919753 | 06 November 2026
Project repository: github.com/hafolabi/Oladipo_Afolabi_301919753

1. Introduction

I selected AfolabiPay, a fictional fintech platform, as the case study for this Advanced Database Systems project. It provided a practical way to explore how databases support deposits, withdrawals, transfers, account balances and payment history. The work covers five related activities. PostgreSQL was used for the main financial records, while MongoDB provided a document-based comparison. All customers and payments are fictional. Amounts were recorded in kobo (100 kobo = NGN 1), and balances were calculated from completed transaction entries rather than stored as editable figures.

2. Database Design and Modelling

The relational database was designed around five PostgreSQL tables: customers, accounts, transactions, transaction_entries and audit_events. A customer can have several accounts, and each payment is linked to entries showing the movement of funds. Audit events keep a record of important payment actions.

Primary and foreign keys were used to maintain the relationships between records. The transaction_entries table has a combined key made up of the transaction ID and account ID. Unique constraints, required fields and validation rules help prevent duplicate or incorrect records. The design also follows the principles of normalisation (1NF, 2NF and 3NF) by separating customer, account and payment information and avoiding unnecessary repetition (PostgreSQL Global Development Group, n.d.-a).

Test results: The five tables were populated with 4 customers, 5 accounts, 7 transactions, 10 entries and 8 audit events. All six attempts to save invalid data were rejected. The closing balances calculated from the entries were NGN 8,000 for Ada and NGN 7,000 for Bello.

3. Query Processing and Optimisation

Four SQL queries were written to retrieve account balances, payment history, daily transfer totals and payment problems. To find out whether history searches could run faster, I tested an index on account ID and transaction ID. The same query and data were used before and after adding the index (PostgreSQL Global Development Group, n.d.-c, n.d.-e).

Measure	Before indexing	After indexing
Payment-history rows	203	203 (same results)
Median query time (five runs)	44.248 ms	1.971 ms
Additional index storage	None	1,256 kB

The index reduced the median query time from 44.248 ms to 1.971 ms, an improvement of about 22.45 times. Both tests returned the same 203 rows. However, the index required an additional 1,256 kB of storage and must be maintained as records change. The test used 40,410 entries in PGlite 0.5.12 / PostgreSQL 18.6, so these timings describe the test environment rather than expected production performance.

4. Transactions and Concurrency Control

Transaction tests were carried out on PostgreSQL 18.4 to see what happens when a payment succeeds, fails or overlaps with another request. I used separate database connections to check these situations. Before completing a transfer, the relevant accounts were locked, the available funds were checked, and the payment, its two entries and an audit event were recorded. A successful transfer was committed, while an error caused the transaction to roll back (PostgreSQL Global Development Group, n.d.-d).

Test	Observed result
Commit	A NGN 2,000 transfer reduced Ada's balance from NGN 10,000 to NGN 8,000 and increased Bello's from NGN 0 to NGN 2,000.
Rollback	A deliberate error cancelled the operation. No new transaction, entry or audit event was saved, and balances did not change.
Read Committed	One connection could not see another connection's unfinished payment, but could see it after it was committed.
Concurrent withdrawals	Two requests each attempted to withdraw NGN 800 from NGN 1,000. One succeeded; the other waited, saw NGN 200 remaining and was rejected.

All four checks passed. In the competing withdrawal test, the second request had to wait until the first was complete. It was then rejected because only NGN 200 remained. The result showed why funds should be checked after locking the account, provided that all payment operations follow the same locking process.

5. NoSQL and Document-Based Databases

For the NoSQL activity, I created two MongoDB collections: customer_profiles and transaction_events. Customer details were grouped in profile documents, while each payment event stored its two accounting entries and references to the relevant customers. This structure keeps related event information together without requiring table joins (MongoDB, Inc., n.d.-a, n.d.-b).

In the sample transfer TRF-001, account 1000000001 was debited by 200,000 kobo and account 1000000002 was credited by the same amount. BSON Long integers were used to preserve exact money values. The sample database contained two profiles and three payment events. The queries returned customer information by email, an account's history and completed transfers, as intended.

MongoDB was useful for organising flexible customer and payment documents, but its references do not provide the same foreign-key checks as PostgreSQL. Saving a single event atomically also does not, on its own, prevent overdrafts across different payments. PostgreSQL therefore remained the main financial record in the proposed system. Automatic synchronisation between the two databases was not implemented.

6. Running the Project

The application can be run from the project repository using npm run setup followed by npm start. Its menu allows sample payments and account-history searches. The README explains the steps, and the saved checks can be repeated using npm run check:p1 through npm run check:p4. Restarting the application resets the sample data.

7. Distributed and Cloud Database Proposal

For the final activity, I proposed a cloud database arrangement with a PostgreSQL primary and standby in separate availability zones, together with three MongoDB replica-set members across three zones. This part remained a design proposal; no cloud deployment or failover test was carried out.

Synchronous PostgreSQL replication could allow the primary database to wait until a standby has recorded a committed change. This would reduce the risk of losing an acknowledged payment, although it could increase delays when the standby is slow or unavailable. A failover process would also need to prevent two servers from accepting writes as the primary (PostgreSQL Global Development Group, n.d.-b).

MongoDB replication could improve availability when a server fails, but secondary reads may be delayed and writes may pause if a majority of members is unavailable (MongoDB, Inc., n.d.-c, n.d.-d). Payment decisions would therefore continue to use the PostgreSQL primary and the account-locking method tested earlier. Sharding was not proposed for this small dataset because dividing the ledger across servers would add complexity to transfers.

8. Reflection and Conclusion

This project helped me understand that a payment database must do more than store transactions. The records must remain accurate, every debit must have a matching credit, and withdrawals must be checked before they are approved. The concurrency test made this especially clear: two requests can reach the same account almost at once, but the database must still prevent them from spending the same funds.

I also learned that a faster query is only an improvement when it returns the correct results. The indexing test made it possible to compare speed while considering the extra storage required. Based on these findings, PostgreSQL is the better fit for the main payment ledger, while MongoDB can support more flexible customer and event records. The cloud design is a possible next step, but it has not yet been implemented.

Full technical files and supporting outputs are available in the project repository.

References

MongoDB, Inc. (n.d.-a). *MongoDB BSON types*. https://www.mongodb.com/docs/manual/reference/bson-types/

MongoDB, Inc. (n.d.-b). *Embedded data in your MongoDB schema*. https://www.mongodb.com/docs/manual/data-modeling/embedding/

MongoDB, Inc. (n.d.-c). *Replication and replica set fundamentals*. https://www.mongodb.com/docs/manual/replication/

MongoDB, Inc. (n.d.-d). *Write concern*. https://www.mongodb.com/docs/manual/reference/write-concern/

PostgreSQL Global Development Group. (n.d.-a). *Constraints*. In *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/ddl-constraints.html

PostgreSQL Global Development Group. (n.d.-b). *Log-shipping standby servers*. In *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/warm-standby.html

PostgreSQL Global Development Group. (n.d.-c). *Multicolumn indexes*. In *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/indexes-multicolumn.html

PostgreSQL Global Development Group. (n.d.-d). *Transaction isolation*. In *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/transaction-iso.html

PostgreSQL Global Development Group. (n.d.-e). *Using EXPLAIN*. In *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/using-explain.html